

文本匹配 (Text Matching)

意图识别 · 语义检索 · 问答匹配 · 大模型应用

Part 1 业务场景

1.1 什么是文本匹配

定义：给定文本对 (A, B)，计算其语义相似度或判断相关性，输出相似分数或类别标签。

示例：

- 文本 A: "今天天气不错"
- 文本 B: "天气真好啊"
- 匹配模型输出: 相似度 0.92

核心问题：语义是否等价 / 相关 / 蕴含？

输出形式：相似分数 (0~1) 或分类标签 (相似/不相似/矛盾)

核心地位：搜索、推荐、问答、对话系统的基础组件

新手解释 - "蕴含"：在自然语言推理 (NLI) 中，"蕴含"指文本A的语义可以推导出文本B的语义。例如 A="他正在跑步" 蕴含 B="他在运动"，但 A 不蕴含 B="他在跑步机上跑"。

1.2 主流业务场景

场景	文本 A	文本 B	示例
意图识别	用户输入	意图模板	"帮我订机票" → 订票意图
FAQ问答检索	用户提问	知识库问题	"怎么退款" → 退款FAQ
语义搜索	查询词	文档标题/正文	"Python入门" → 相关教程
NLI自然语言推理	前提句	假设句	"猫在垫子上" vs "动物在休息"
文本去重/聚类	文本 A	文本 B	新闻去重、评论聚类、相似问去重

新手解释 - 文本去重/聚类：当你有大量文本需要"合并相同意思的"时，就是文本去重。例如搜索引擎中，用户问"怎么退款"和"如何退钱"本质是同一个问题，应该合并为一个FAQ条目而不是分别回答。文本去重本质上也是两两做文本匹配，再根据相似度决定是否合并。

新手解释 - NLI (Natural Language Inference, 自然语言推理)：判断两个句子之间的逻辑关系，分为三类：

- 蕴含 (Entailment)：**A为真时B一定为真。例：A="小明在吃饭" → B="小明在吃东西"
- 矛盾 (Contradiction)：**A为真时B一定为假。例：A="小明在吃饭" → B="小明在睡觉"
- 中立 (Neutral)：**A为真时B可能真也可能假。例：A="小明在吃饭" → B="小明在吃面条"

1.3 技术选型关键因素

- **数据量**：有标注 vs 无标注
- **延迟要求**：实时 vs 离线
- **候选集规模**：千级 vs 亿级
- **精度要求**：粗排 vs 精排

1.4 核心概念：候选集 (Candidate Set)

新手解释 - 候选集是什么？
候选集就是在文本匹配任务中，需要与查询文本进行比对的所有目标文本的集合。简单说，就是"你有一堆文本，用户来了一个查询，你要从这堆文本里找到最匹配的"——这堆文本就是候选集。

举例：假设你开发了一个智能客服FAQ系统，知识库里有 10,000 个标准问题。用户输入"怎么退款"，你需要从这 10,000 个问题中找到最匹配的那个。这 10,000 个问题就是**候选集**。

候选集规模对技术选型的影响：

候选集规模	典型场景	推荐方法	为什么
百级 (<500)	意图分类、小规模FAQ	CrossEncoder 直接精排	延迟可接受，精度优先
千级 (500~5,000)	中等FAQ、文档检索	BiEncoder + CrossEncoder	先粗排缩小范围再精排
万级 (5,000~100,000)	大FAQ、商品搜索	BiEncoder + FAISS (ANN)	必须用向量检索，无法逐对比较
百万~亿级	搜索引擎、推荐系统	BiEncoder + FAISS + 精排	两阶段流水线是唯一可行方案

为什么候选集规模这么重要？

假设单次 CrossEncoder 推理耗时 10ms：

候选集大小	总耗时	能否接受
100	1 秒	勉强
1,000	10 秒	不可接受
10,000	100 秒	完全不可接受
1,000,000	~3 小时	不可能

这就是为什么候选集越大，越需要 BiEncoder 做**粗召回**——先用向量检索把候选集从百万级缩小到百级，再用 CrossEncoder 精排。

候选集在实际业务中的角色：





候选集的两个关键属性：

- **静态候选集：**知识库、FAQ等固定不变的内容，向量可提前计算并缓存（BiEncoder 的优势）
- **动态候选集：**实时变化的新闻、商品等，需要定期更新索引

新手记忆口诀：候选集就是你"大海捞针"的那片"海"。海越大，越需要好的工具（BiEncoder），否则永远捞不到针。

Part 2 核心方法

2.1 传统方法

2.1.1 编辑距离 (Levenshtein Distance)

将字符串 A 通过插入、删除、替换操作变成字符串 B 所需的最少操作次数。

示例：

- A = "kitten", B = "sitting"
- k→s（替换），e→i（替换），插入 g → 编辑距离 = 3

新手解释：编辑距离衡量的是两个字符串在"字面"上的差异，不理解语义。比如"高兴"和"伤心"编辑距离很小（只改一个字），但语义完全相反；而"开心"和"高兴"编辑距离大，但语义相同。因此编辑距离只适合拼写纠错等字面匹配场景。

2.1.2 TF-IDF

新手解释 - TF-IDF (Term Frequency - Inverse Document Frequency)：

- **TF (词频)：**某个词在当前文档中出现的频率。出现越多，越重要。
- **IDF (逆文档频率)：**某个词在所有文档中出现的广泛程度。越常见（如的""是"），IDF越低，区分度越差。
- **TF-IDF = TF × IDF：**一个词在当前文档频繁出现，但在其他文档很少出现，说明它对当前文档很有代表性。

逐步计算示例：

假设有3篇文档：

- D1: "深度学习 与 机器 学习" (4个词)
- D2: "深度学习 图像 识别" (3个词)
- D3: "统计 学习 理论" (3个词)

查询词: "深度学习"

文档	TF	df	IDF = $\log(3/df)$	TF-IDF
D1	$1/4 = 0.25$	2	$\log(3/2) = 0.41$	0.10
D2	$1/3 \approx 0.33$	2	$\log(3/2) = 0.41$	0.14
D3	$0/3 = 0.00$	—	—	0.00

对比词: "统计" (仅出现在D3)

- $TF(D3) = 1/3 \approx 0.33$, $df=1$, $IDF = \log(3/1) = 1.10$
- $TF-IDF = 0.33 \times 1.10 = 0.36$ (远高于"深度学习"!)

结论: 稀有词自动获得更高权重——这正是IDF的设计目的。"深度学习"出现在2/3的文档中, 不够稀有; "统计"只在1篇中出现, 区分度更高。

代码示例:

```
from sklearn.feature_extraction.text import TfidfVectorizer

docs = ["深度学习 与 机器 学习", "深度学习 图像 识别", "统计 学习 理论"]
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(docs)

# 计算两两余弦相似度
from sklearn.metrics.pairwise import cosine_similarity
sim = cosine_similarity(tfidf_matrix)
# sim[i][j] = 文档i与文档j的TF-IDF余弦相似度
```

2.1.3 BM25

TF-IDF 的改进版本, 加入了词频饱和度和文档长度归一化。

新手解释 - BM25: BM25 是搜索引擎中最常用的排序算法。相比 TF-IDF, 它有两个改进:

1. **词频饱和:** 一个词出现10次不一定比出现5次要重要2倍, BM25让词频的收益递减
2. **文档长度归一化:** 长文档更容易匹配到查询词, BM25对长文档做惩罚

BM25公式:

$$BM25(q, d) = \sum IDF(q_i) \times [f(q_i, d) \times (k_1 + 1)] / [f(q_i, d) + k_1 \times (1 - b + b \times |d| / avgdl)]$$

参数含义:

- $f(q_i, d)$: 词 q_i 在文档 d 中的词频
- $|d| / avgdl$: 文档长度 / 语料平均长度
- $k_1 \in [1.2, 2.0]$: TF饱和速度 (推荐1.5)
- $b \in [0, 1]$: 长度归一化强度 (推荐0.75)

示例: 搜索"深度学习入门", BM25会综合考虑: 词频 (文档中包含多少个搜索词)、文档长度 (短文档匹配到更珍贵)、词的稀有度 ("深度学习"比"入门"更有区分度)

代码示例:

```
from rank_bm25 import BM25Okapi

corpus = [doc.split() for doc in docs] # 每条文档分词后的列表
bm25 = BM25Okapi(corpus)
scores = bm25.get_scores(query.split()) # 返回每条文档的相关性得分
```

面试：TF-IDF和BM25的代码

2.1.4 Word2Vec / GloVe 均值向量

将每个词映射为向量，然后对句子中所有词向量取平均，用余弦相似度比较。

新手解释 - Word2Vec: Word2Vec 是一种将词语转化为数值向量的方法。核心思想：**上下文相似的词，语义也相似**。例如"国王"和"女王"的向量很接近，因为它们经常出现在相似的上下文中。

新手解释 - GloVe: GloVe (Global Vectors) 是另一种词向量方法，基于全局词共现统计。与 Word2Vec 从局部上下文窗口学习不同，GloVe 利用整个语料库中词与词共同出现的频率矩阵来学习。

均值向量是如何计算的？

假设每个词用一个 3 维向量表示（真实场景通常是 100~300 维）：

句子："今天 天气 不错"（分词后 3 个词）

词	向量
今天	[0.2, 0.5, 0.1]
天气	[0.8, 0.3, 0.6]
不错	[0.4, 0.7, 0.2]

Step 1: 按位求和

```
[0.2, 0.5, 0.1]
[0.8, 0.3, 0.6]
[0.4, 0.7, 0.2]
-----
[1.4, 1.5, 0.9] ← 逐位相加
```

Step 2: 除以词数（取平均）

```
[1.4/3, 1.5/3, 0.9/3] = [0.47, 0.50, 0.30]
```

最终：句子向量 = [0.47, 0.50, 0.30]（3个词向量的算术平均）

Step 3: 用余弦相似度比较两个句子

句子A向量 = [0.47, 0.50, 0.30]
 句子B向量 = [0.52, 0.45, 0.35]

$$\begin{aligned}
 \text{余弦相似度} &= (0.47 \times 0.52 + 0.50 \times 0.45 + 0.30 \times 0.35) \\
 &\quad / (\sqrt{(0.47^2 + 0.50^2 + 0.30^2)} \times \sqrt{(0.52^2 + 0.45^2 + 0.35^2)}) \\
 &= 0.5744 / (0.745 \times 0.771) \\
 &= 0.5744 / 0.574 \\
 &\approx 0.999 \leftarrow \text{非常相似}
 \end{aligned}$$

代码示例：

```

import numpy as np
from gensim.models import word2vec

def sent_vec(tokens, model):
    """将句子所有词向量取平均，得到句子向量"""
    vecs = [model.wv[w] for w in tokens if w in model.wv]
    return np.mean(vecs, axis=0) # axis=0 表示按列（每个维度）取平均

# 示例
tokens = ["今天", "天气", "不错"]
sentence_vector = sent_vec(tokens, model) # 形状：(300,) 即300维向量
  
```

均值向量的问题："我不开心"和"我很开心"均值向量可能很接近（因为大部分词相同），但语义相反。这就是均值向量的根本缺陷——丢失了词序和语义结构信息。

传统方法对比：

方法	原理	优点	缺点
编辑距离	字符操作次数	简单快速	只看字面，不懂语义
TF-IDF	词频×逆文档频率	考虑词的重要性	无语义理解
BM25	改进的TF-IDF	搜索引擎标配	无语义理解
词向量均值	词向量平均+余弦相似度	有一定语义能力	丢失词序信息

2.1.5 Word2Vec 的核心局限

虽然 Word2Vec 让机器"读懂"了词语之间的关系，但它在文本匹配任务中有**三大致命缺陷**：

局限一：一词一义（无法处理多义词）

Word2Vec 给每个词分配一个固定的向量，但在真实语言中，同一个词在不同上下文里意思完全不同：

词	上下文	含义	Word2Vec 的问题
"苹果"	"我喜欢吃苹果"	水果	同一个向量，无法区分
"苹果"	"苹果发布了新手机"	科技公司	同一个向量，无法区分
"银行"	"我去银行取钱"	金融机构	同一个向量，无法区分
"银行"	"河边的银行"	河岸	同一个向量，无法区分

这就是 BERT 引入上下文相关编码的原因——同一个"苹果"，在不同句子里会得到不同的向量。

局限二：静态向量，无法处理新词（OOV 问题）

OOV = Out-Of-Vocabulary（词表外词汇），指的是模型训练时从未见过的词。

为什么会有 OOV？

Word2Vec 训练时会先构建一个**固定词表**（比如最常见的 100 万个词）。训练完成后，这个词表就锁死了——任何不在词表中的词都是 OOV，模型完全无法处理。

词表（训练时见过）	词表外（OOV）
我、你、苹果、银行、...	ChatGPT
深度学习、自然语言、...	Transformer
（共计 100 万个词）	元宇宙、NFT、web3...
	拼写错误：recieve（应为receive）
	专有名词：张三丰、特拉法尔加广场
	新造词：躺平、内卷（旧词新义也算）

OOV 在实际业务中的严重性：

场景	输入	OOV 词	后果
智能客服	"我的 AirPods Pro 2 连不上"	AirPods, Pro	产品名全丢，无法匹配 FAQ
法律检索	"《个人信息保护法》第13条"	个人信息保护法	法规名丢失，检索失败
医疗问答	"布洛芬和阿莫西林能一起吃吗"	布洛芬, 阿莫西林	药名全部 OOV，匹配无意义
金融风控	"ETH 合约地址 0x7a25..."	ETH, 0x7a25...	地址信息完全丢失

三种解决方案的对比：

方案	原理	例子	优缺点
WordPiece / BPE (BERT 使用)	拆成子词	Transformer → Transform + ##er	无限词表，子词有语义
字符级编码 (ELMo 的 CNN)	拆成字母	ChatGPT → C,h,a,t,G,P,T	永远不 OOV，但语义弱
FastText (Word2Vec 改进版)	n-gram 子词	apple → <ap, app, ppl, ple, le>	可处理拼写错误，但仍是静态向量

一句话总结：Word2Vec 像一本字典——只认识印在书里的词；BERT 的 WordPiece 像会拆字——遇到"饕餮"不认识，但拆成偏旁部首也能猜个大概。

局限三：均值池化丢失词序和语义结构

把句子所有词向量平均，等于把词序信息彻底丢弃：

句子A: "我 爱 你" → mean([我, 爱, 你]) = v1
句子B: "你 爱 我" → mean([你, 爱, 我]) = v1 (完全相同!)
句子C: "我 不 爱 你" → mean([我, 不, 爱, 你]) ≈ v1 (非常接近!)

三个语义完全不同的句子，向量几乎一样。这就是为什么文本匹配需要更强大的模型。

局限四：训练数据依赖严重

问题	说明
领域漂移	用新闻语料训练的 Word2Vec, "卷积"理解为数学运算, 但在医疗文本中表示"卷积神经网络"
语料规模	小规模语料训练的向量质量差, 稀有词向量不可靠
语言差异	中文需要分词预处理, 分词错误会导致向量质量下降

总结：Word2Vec → BERT 的演进逻辑

word2Vec 时代		BERT 时代
一词一向量	→	一词多向量（上下文相关）
OOV 崩溃	→	wordPiece 子词拆分
词序丢失	→	Transformer 自注意力
静态特征	→	动态特征（12层隐状态）

给新手的建议：理解 Word2Vec 的局限，才能真正理解为什么 BERT 是革命性的。Word2Vec 教会了机器"词是什么"，BERT 教会了机器"在这个语境里这个词是什么意思"。

2.2 神经网络方法

2.2.1 BiEncoder（双塔模型 / 表示型模型）

两个独立的编码器分别将文本 A 和文本 B 编码为向量，再计算向量相似度。

文本A → [Encoder] → 向量a
文本B → [Encoder] → 向量b
→ cosine(a, b) → 相似度

新手解释 - BiEncoder：想象两个人分别阅读两篇文章后各自写一个摘要，然后比较两个摘要的相似度。优点是两个编码器独立工作，向量可以预先计算并缓存，适合大规模检索。缺点是两篇文章在编码时"互不知道对方的存在"，无法捕捉细粒度的交互特征。

关键特性：

- 向量可预计算，检索时只需计算相似度
- 适合大规模召回（百万~亿级）
- 缺点：交互不充分，精度有限

新手解释 - "召回"是什么？

"召回"（Recall/Retrieval）在搜索和推荐系统中指的是从海量候选集中快速筛选出一小批可能相关的条目。它不要求精挑细选，只要求"别漏掉重要的"。

类比：你要从 100 万人的图书馆里找到喜欢读科幻小说的人——

- **召回阶段**：快速扫一眼所有人的借阅记录，筛出借过科幻类书籍的 1000 人（快但粗糙）
- **精排阶段**：仔细分析这 1000 人的借阅历史、评分、阅读频率，排出 Top 10（慢但精准）



"召回"和"精排"的核心区别：

	召回	精排
目标	宁可多捞，不能漏掉	精准排序，优中选优
候选规模	百万~亿级	百~千级
速度要求	毫秒级	可接受几十毫秒
精度要求	召回率优先（查全）	准确率优先（查准）
典型方法	BiEncoder + FAISS	CrossEncoder

新手记忆：召回 = 海选（快而不精），精排 = 决赛（慢而精准）。

2.2.2 CrossEncoder（交叉编码器 / 交互型模型）

将文本 A 和文本 B 拼接后一起输入编码器，在编码过程中进行深度交互。

[CLS] 文本A [SEP] 文本B [SEP] → [Encoder] → 分类/分数

新手解释 - CrossEncoder：想象两个人一起阅读两篇文章，边读边讨论异同点。因为两篇文章同时输入模型，每个词都可以"看到"另一篇文章的所有词，交互更充分，精度更高。但代价是每次只能处理一对文本，无法预计算向量，速度慢，不适合大规模检索。

关键特性：

- 深度交互，精度高
- 无法预计算向量，速度慢
- 适合小候选集精排（百~千级）

代码示例：

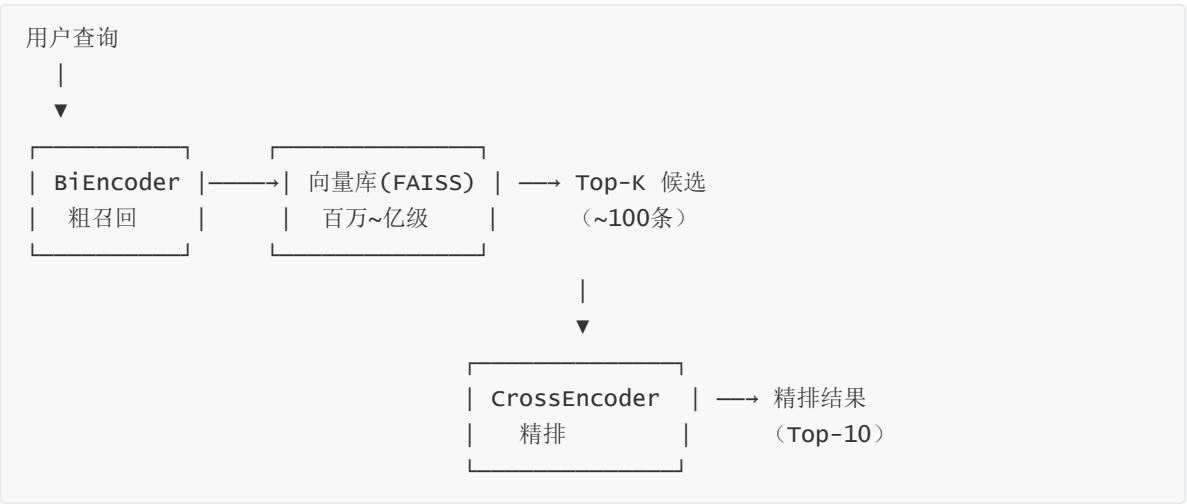
```
from sentence_transformers import CrossEncoder

ce = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')
pairs = [['文本A', '文本B']]
scores = ce.predict(pairs) # 输出相关性分数
```

2.2.3 BiEncoder vs CrossEncoder 对比

维度	BiEncoder	CrossEncoder
交互方式	编码后交互（晚期交互）	编码中交互（早期交互）
精度	中高	高
速度	快（向量可缓存）	慢（每对都要算）
适用场景	大规模召回	小候选集精排
典型应用	语义搜索、FAQ检索	重排序、NLI判断

2.2.4 两阶段检索架构（工业标准方案）



新手解释 - 为什么需要两阶段？ 这就像招聘流程：第一步HR快速筛选简历（BiEncoder粗召回），从上万份简历中选出100份；第二步面试官深度评估（CrossEncoder精排），从100人中选出最终人选。如果直接让面试官看上万份简历，效率太低；如果只让HR做最终决定，精度不够。

两阶段代码示例：

```
# Stage 1: BiEncoder 粗召回
query_vec = bi_model.encode(query) # 编码查询
top_k = faiss_index.search(query_vec, k=100) # ANN 检索 Top-100

# Stage 2: CrossEncoder 精排
pairs = [[query, docs[i]] for i in top_k] # 构造句对
scores = cross_model.predict(pairs) # 逐对打分
final_results = argsort(scores)[:10] # 取 Top-10
```

2.2.5 SBERT (Sentence-BERT)

SBERT 是对 BERT 的改进，专门用于生成高质量的句子向量。它使用 Siamese/Triplet 网络结构微调 BERT，使其输出的句子向量在语义空间中有意义。



新手解释 - SBERT：原始 BERT 直接取句子向量效果很差（任意两句话的余弦相似度都很高），因为 BERT 预训练的目标不是生成好的句子表示。SBERT 通过对比学习/分类任务微调 BERT，让语义相似的句子向量靠近、不相似的远离，从而得到有意义的句子嵌入。

新手解释 - Pooling (池化)： BERT 输出每个 token 的向量，Pooling 是将这些向量合并为一个句子向量的方法：

- **CLS池化：** 取 [CLS] token 的向量作为句子表示 (最常用)
- **平均池化 (Mean Pooling)：** 对所有 token 向量取平均 (SBERT推荐)
- **最大池化 (Max Pooling)：** 取每个维度的最大值

关键特性：

- 本质上是 BiEncoder 架构，但通过对比学习微调获得高质量向量
- 推理速度与 BiEncoder 相同，向量可预计算
- 是目前最常用的句子嵌入基线模型

代码示例：

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')

u = model.encode('今天天气不错')
v = model.encode('天气真好啊')
sim = cosine_similarity([u], [v])[0][0] # 约 0.92
```

2.2.6 ColBERT (晚期交互模型)

ColBERT 介于 BiEncoder 和 CrossEncoder 之间，保留了 token 级别的交互，同时支持预计算。

文本A → [Encoder] → [a1, a2, ..., an] $\neg$
 $\mapsto$ MaxSim $\rightarrow$ 相似度
文本B → [Encoder] → [b1, b2, ..., bm] $\neg$

新手解释 - ColBERT：

- BiEncoder 只比较两个"摘要" (句子向量)，丢失细节
- CrossEncoder 逐词对比，精度高但太慢
- ColBERT 折中：每个词都保留自己的向量，然后用 **MaxSim** 操作计算相似度

MaxSim 操作： 对于文本A的每个词向量，找到文本B中与它最相似的词向量，取这些最大相似度的平均值。就像对A中的每个词，在B中找"最匹配的词"，然后汇总。

示例： A="苹果手机价格", B="iPhone多少钱"

- "苹果" 最匹配 "iPhone" (相似度0.9)
- "手机" 最匹配 "iPhone" (相似度0.7)
- "价格" 最匹配 "多少钱" (相似度0.85)
- ColBERT得分 = (0.9+0.7+0.85)/3 = 0.82

三种架构对比：

- `margin=0`：最严格，要求负例 $\cos < 0$ （正交即可），适用于简单任务
- `margin=0.3`：常用默认值，适合大多数场景
- `margin=0.5`：更宽松，允许负例有较高相似度，适用于难区分场景

训练数据构建方式：

数据来源	正例	负例
STS数据集	分数 ≥ 4	分数 ≤ 1
NLI	蕴含对	矛盾对
FAQ业务数据	同问题变体	不同问题

代码示例：

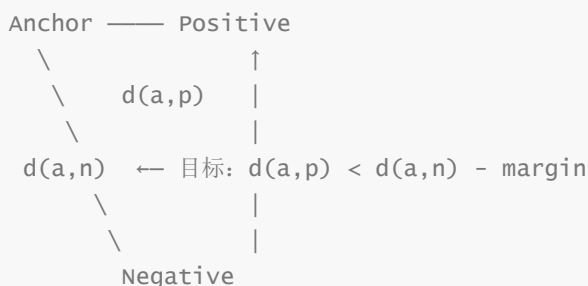
```
loss_fn = nn.CosineEmbeddingLoss(margin=0.3)
# label: +1 (正例) 或 -1 (负例)
loss = loss_fn(emb_a, emb_b, label)
```

TripletLoss：三元组损失，要求锚点与正例的距离 $<$ 锚点与负例的距离 - margin。

新手解释 - TripletLoss：

- **锚点 (Anchor)**：参考样本，如"帮我订机票"
- **正例 (Positive)**：与锚点语义相似的样本，如"我要买飞机票"
- **负例 (Negative)**：与锚点语义不同的样本，如"今天天气怎么样"
- **目标**：让 锚点-正例 的距离 比 锚点-负例 的距离 至少小 margin
- **margin**：一个安全间距，确保正例和负例之间有足够的"间隔"

几何直觉：



Loss 解读：当 $d(a,p) < d(a,n) - \text{margin}$ 时， $\text{Loss}=0$ （已满足要求）；否则对 $d(a,p)$ 过大或 $d(a,n)$ 过小都产生惩罚。

代码示例：

```
loss_fn = nn.TripletMarginLoss(margin=0.5, p=2)
loss = loss_fn(anchor_vec, positive_vec, negative_vec)
```

InfoNCE / NT-Xent：对比学习中最常用的损失函数。

新手解释 - InfoNCE (Information Noise Contrastive Estimation)：

想象一个"认人游戏"：在一群人中找到你的朋友。InfoNCE就是让模型从一堆"干扰项"（负例）中正确识别出"目标"（正例）。

NT-Xent (Normalized Temperature-scaled Cross Entropy)：InfoNCE的改进版，加入了温度参数 τ (temperature)：

- τ 越小，模型越关注"难区分"的负例（更严格）
- τ 越大，模型对负例更"宽容"（更宽松）

直觉理解：温度就像考试难度——低温=难题模式（严格区分），高温=简单模式（宽松判断）

2.4 评估指标

文本匹配任务需要用合适的指标评估模型效果，不同场景适用不同指标。

2.4.1 分类指标（适用于 NLI / 相似/不相似 判断）

指标	含义	适用场景
Accuracy	预测正确的比例	正负样本均衡时
Precision	预测为正的样本中，真正为正的的比例	关注"预测对的占多少"
Recall	真正为正的样本中，被预测为正的的比例	关注"漏掉了多少"
F1 Score	Precision 和 Recall 的调和平均	正负样本不均衡时
AUC	ROC曲线下面积，衡量排序能力	不依赖阈值的整体评估

新手解释 - Precision vs Recall：

假设有100条查询，其中20条是"订票"意图：

- 模型预测了15条为"订票"，其中12条预测对了 → $\text{Precision} = 12/15 = 80\%$
- 实际20条"订票"中，模型只找出了12条 → $\text{Recall} = 12/20 = 60\%$
- **Precision 回答：**"我说的是，有多少真是？"
- **Recall 回答：**"真正是的，我找出了多少？"

新手解释 - F1 Score：Precision 和 Recall 的平衡指标。 $F1 = 2 \times P \times R / (P + R)$ 。当 $P=80\%$, $R=60\%$ 时， $F1 = 2 \times 0.8 \times 0.6 / (0.8 + 0.6) \approx 68.6\%$ 。

新手解释 - AUC：AUC 衡量模型"把正例排在负例前面"的能力。AUC=0.5 表示随机猜，AUC=1.0 表示完美排序。AUC 不受阈值和样本比例影响，是不均衡数据下最可靠的指标。

2.4.2 排序指标（适用于检索 / 召回场景）

指标	含义	适用场景
MRR	第一个正确结果的排名倒数均值	只关心第一个正确结果
MAP	各查询的AP平均值，AP是Precision-Recall曲线下面积	关心所有正确结果的排序
NDCG@K	考虑位置权重的增益，前K个结果的排序质量	相关性有程度差异
Hit@K / Recall@K	前K个结果中是否命中 / 命中比例	召回效果评估

新手解释 - MRR (Mean Reciprocal Rank)：

对于每个查询，找到第一个正确结果的排名 r ，得分 = $1/r$ 。所有查询取平均。

- 查询1：正确结果排第1位 → $1/1 = 1.0$
- 查询2：正确结果排第3位 → $1/3 = 0.33$
- 查询3：正确结果排第2位 → $1/2 = 0.5$

- $MRR = (1.0 + 0.33 + 0.5) / 3 = 0.61$

新手解释 - NDCG (Normalized Discounted Cumulative Gain) :

- 核心理念：**排在前面的结果更重要**，位置越靠后折扣越大
- "Discounted": 排名越靠后，贡献打折越多（如第1位权重=1，第2位=1/2，第3位=1/3...）
- "Normalized": 除以理想排序的得分，使结果在0~1之间
- $NDCG@10 = 0.9$ 表示前10个结果的排序质量达到了理想排序的90%

新手解释 - Hit@K / Recall@K:

- **Hit@K**: 前K个结果中是否至少有一个正确的。Hit@10=0.8 表示80%的查询在前10个结果中找到了正确答案
- **Recall@K**: 前K个结果中找到了多少正确答案。如果有5个正确答案，Recall@10=0.6 表示在前10个中找到了3个

2.4.3 回归指标 (适用于 STS 语义相似度评分)

指标	含义
Spearman相关系数	预测分数与真实分数的排序相关性
Pearson相关系数	预测分数与真实分数的线性相关性

新手解释: STS任务中，每对文本有一个0~5的相似度评分。Spearman衡量"模型打分的高低顺序是否和人工一致"，Pearson衡量"模型打分的线性关系"。实践中 Spearman 更常用，因为它对异常值更鲁棒。

2.5 相似度计算方式

向量之间的相似度有多种计算方式，选择哪种直接影响匹配效果。

2.5.1 三种常用方式

方式	公式	特点
余弦相似度	$\cos(a,b) = (a \cdot b) / (a b)$	只看方向，不看大小，范围[-1,1]
点积 (内积)	$a \cdot b = \sum (a_i \times b_i)$	方向+大小都考虑
欧氏距离	$d(a,b) = \sqrt{\sum (a_i - b_i)^2}$	绝对距离，值越小越相似

新手解释 - 三种方式的区别:

想象两个箭头:

- **余弦相似度**: 只看两个箭头"指的方向是否一致"，不管箭头多长。适合只关心语义方向、不关心向量幅度的场景
- **点积**: 方向一致且箭头都很长时，点积大。既考虑方向又考虑大小
- **欧氏距离**: 两个箭头尖端的直线距离。值越小越相似

关键关系: 当向量做了L2归一化 ($|a|=1, |b|=1$) 后，**余弦相似度 = 点积**。因此实践中通常先归一化，然后用点积计算，效率更高。

2.5.2 向量归一化

新手解释 - 为什么要归一化？

- 归一化后，余弦相似度 = 点积，计算更快
- 归一化消除了向量长度的影响，使相似度只取决于语义方向
- FAISS 等向量库在归一化后可使用内积索引，效率更高

L2归一化：向量除以其L2范数（长度）。 $a_{\text{normalized}} = a / |a|$

2.5.3 如何选择？

场景	推荐方式	原因
语义相似度	余弦相似度	只关心方向，文本长度不影响
向量检索（FAISS）	归一化 + 点积	等价余弦且计算更快
聚类分析	欧氏距离	保留绝对距离信息

Part 3 训练与落地

3.1 训练常见问题

问题1：难负例缺失

问题描述：随机负例太简单，模型学不到细粒度区分能力。

新手解释 - 难负例（Hard Negative）：

- **简单负例：**与查询完全不相关的样本，如"订机票" vs "做红烧肉"，模型轻松区分
- **难负例：**与查询表面相似但语义不同的样本，如"订机票" vs "退机票"，模型容易混淆

如果训练数据全是简单负例，模型就像只做"1+1=2"级别的题，遇到稍难的题就不会了。

解决思路：

- BM25 检索的 Top-K 作为难负例（字面相似但语义不同的样本）
- 模型预测高分但标注为负的样本（模型最容易犯错的样本）
- In-batch negatives 提供中难度负例

新手解释 - In-batch negatives：在一个训练batch中，每个样本的正例是它自己对应的配对，而batch中其他所有样本自动作为负例。这样不需要额外构造负例，训练效率高。

问题2：样本不均衡

问题描述：正例稀少，业务正负比约 1:99，模型倾向于预测全负，指标虚高。

新手解释：假设99%的样本都是负例，模型只要"无脑"全部预测为负例，准确率就有99%！但这个模型毫无用处——它一个正例都找不到。这就是"指标虚高"。

解决思路：

- 过采样正例 / 下采样负例
- NLI、STS 公开数据弱监督预训练
- Focal Loss 等对难样本加权

新手解释 - Focal Loss：普通损失函数对所有样本一视同仁，Focal Loss 则让模型更关注"难分类"的样本。就像老师让成绩差的学生多做练习，而不是让全班做一样多的题。

新手解释 - 弱监督预训练：用现成的公开数据集（如NLI、STS）先训练一个基础模型，再用业务数据微调。这些公开数据虽然不是你的业务数据，但能帮模型学会基本的语义理解能力。

问题3：领域偏移

问题描述：通用预训练 BERT 与业务文本分布差距大，法律/医疗/金融等垂直领域效果差。

新手解释 - 领域偏移 (Domain Shift)：通用BERT是在维基百科等通用语料上训练的，它理解日常语言，但不理解专业术语。比如在医疗领域，"预后"是一个专业术语（指对疾病发展趋势的预测），但通用模型可能不理解。

解决思路：

- 领域内无监督继续预训练 (MLM)
- 少量有监督微调 (10~100条标注)
- 利用 LLM 生成业务领域伪标签数据

新手解释 - MLM (Masked Language Model)：BERT的预训练方式，随机遮住句子中15%的词，让模型预测被遮住的词。例如"今天[MASK]气不错"→预测"天"。在领域数据上继续做MLM，可以让模型学会领域内的语言模式。

问题4：训练数据不足

问题描述：标注数据有限，模型容易过拟合，泛化能力差。

数据增强方法：

方法	说明	示例
同义词替换	随机替换部分词为同义词	"帮我订机票" → "帮我预订机票"
回译 (Back Translation)	翻译成另一种语言再翻回来	"今天天气好" → (英) "Good weather today" → "今天天气不错"
随机删除	随机删除部分非关键词	"我想订一张去北京的机票" → "我想订去北京的机票"
随机交换	交换相邻词的位置	"订机票帮我" → "帮我订机票"
Dropout增强	SimCSE的方式，同一句子两次Dropout	无监督对比学习的正例构造
LLM生成	用大模型生成同义句	Prompt: "请改写以下句子: 帮我订机票"

新手解释 - 回译 (Back Translation)：这是机器翻译时代留下的经典数据增强方法。把中文翻译成英文，再翻译回中文，得到的句子语义相同但表达不同，天然构成正例对。例如：

- 原句: "这个产品质量很好"
- 回译: "这款商品的品质相当不错"

注意：数据增强要适度，过度增强会引入噪声。建议增强后人工抽检质量。

3.2 推理与部署常见问题

问题1：ANN 召回精度损失

问题描述：FAISS/HNSW 近似搜索存在精确召回率 vs 速度的权衡；数亿向量下精确 Top-K 不可行。

新手解释 - ANN (Approximate Nearest Neighbor, 近似最近邻)：

- 精确搜索：**逐一比较所有向量，保证找到最相似的，但太慢
- 近似搜索：**通过索引结构快速定位"可能"最相似的向量，牺牲少量精度换取巨大速度提升

新手解释 - FAISS：Facebook开发的向量相似度搜索库，支持亿级向量高效检索。

新手解释 - HNSW：Hierarchical Navigable Small World，一种图索引算法。想象一个多层地图：上层是"高速公路"（粗略连接），下层是"小路"（精细连接），搜索时先走高速再走小路，快速定位目标。

解决思路：

- 调节 nprobe (IVF) / ef (HNSW) 参数
- 定期重建索引
- 混用精确+近似策略

新手解释 - nprobe / ef：这两个参数控制搜索的"仔细程度":

- nprobe (IVF索引)：搜索多少个聚类桶，值越大越精确但越慢
- ef (HNSW索引)：搜索多少个邻居节点，值越大越精确但越慢

FAISS 常用索引类型对比：

索引类型	原理	速度	精度	内存	适用规模
Flat	暴力搜索，逐一比较	最慢	完美	大	<10万
IVF	先聚类再搜索相关桶	快	较高	中	10万~千万
IVF+PQ	IVF + 乘积量化压缩	很快	中	小	千万~亿
HNSW	多层图结构导航	快	高	大	百万~千万
IVF+HNSW	IVF内用HNSW做桶内搜索	很快	高	大	千万级

新手解释 - 乘积量化 (PQ, Product Quantization)：

将高维向量切分成多个子段，每段独立聚类编码。例如768维向量切成12段64维，每段用256个聚类中心表示，只需1字节存储。原始768维float需3072字节，PQ压缩后只需12字节，压缩256倍！代价是精度有损失，但通常可接受。

新手解释 - IVF (Inverted File Index)：

先对所有向量做K-Means聚类（如分成1000个桶），搜索时只在与查询最近的nprobe个桶内搜索，跳过大部分无关向量。nprobe越大越精确，但越慢。

选型建议：

- <10万向量：**直接用 Flat，精确且够快
- 10万~百万：**IVF 或 HNSW
- 百万~千万：**IVF+PQ 或 HNSW（内存够的话）
- 千万~亿级：**IVF+PQ，必要时分片

问题2：向量库动态更新

问题描述：文档实时增删，索引需同步更新，全量重建代价高。

解决思路：

- 增量更新索引（FAISS IDMap）
- 设定批量刷新周期 + 实时过滤层

问题3：CrossEncoder 延迟过高

问题描述：候选集 > 200 时，在线精排延迟不可接受；大模型 CrossEncoder 尤为严重。

解决思路：

- 知识蒸馏：用 CrossEncoder 打分训练 MiniLM
- 控制精排候选集大小 (<100)
- 模型量化 (INT8)

新手解释 - 知识蒸馏 (Knowledge Distillation)： 让大模型（教师）的输出作为小模型（学生）的训练目标。就像老师把复杂的知识"提炼"成通俗易懂的形式教给学生，学生虽然不如老师博学，但反应更快。

新手解释 - 模型量化 (Quantization)： 将模型参数从高精度（如FP32，32位浮点数）压缩为低精度（如INT8，8位整数），模型变小变快，精度损失通常很小。就像把高清图片压缩为标清，文件小了很多但看起来差不多。

问题4：冷启动/无标注数据

问题描述：新业务上线，缺少训练对，直接用通用模型效果差。

解决思路：

- 用 LLM 生成 (query, 正例, 负例) 三元组
- 再用这批伪标签训练/微调小模型
- 少量人工校验提升质量

工程经验：部署时建议先 BiEncoder ANN 召回 + 蒸馏后的小型 CrossEncoder 精排，是目前延迟/精度最优的组合方案。

3.3 常见公开数据集

训练文本匹配模型需要大量标注数据，以下是常用的公开数据集，可用于预训练或弱监督学习。

3.3.1 英文数据集

数据集	任务类型	规模	说明
SNLI	NLI（蕴含/矛盾/中立）	57万	斯坦福自然语言推理，最经典的NLI数据集
MNLI	NLI	43万	多领域NLI，覆盖新闻、小说、演讲等
STS-B	语义相似度评分（0~5）	8600对	Semantic Textual Similarity Benchmark
QQP	问题对相似度（二分类）	40万对	Quora问题对，判断两个问题是否等价
Qnli	问答NLI	10万+	判断答案是否回答了问题

3.3.2 中文数据集

数据集	任务类型	规模	说明
LCQMC	问题对相似度	26万对	哈工大大规模中文问题匹配
BQ Corpus	问题对相似度	12万对	银行领域问题匹配
ATEC	问题对相似度	10万对	蚂蚁金服语义相似度
PAWS-X (中文)	释义识别	1万+	含大量"字面相似但语义不同"的难例
ChineseSTS	语义相似度评分	~5000对	中文语义相似度评分数据

新手解释 - 如何使用公开数据集？

1. 预训练阶段：用 SNLI/MNLI 做弱监督预训练，让模型学会基本的语义理解

2. 微调阶段：用业务数据微调，适配具体场景

3. 评估阶段：用 STS-B 等标准数据集做基准测试，对比不同模型

注意：中文场景优先使用中文数据集（LCQMC、BQ Corpus），英文数据集翻译后质量会下降。

3.4 Sentence Transformers 库

Sentence Transformers 是目前最流行的文本匹配工具库，封装了 SBERT、SimCSE 等模型的训练和推理。

3.4.1 核心功能

```
# 安装
# pip install sentence-transformers

from sentence_transformers import SentenceTransformer

# 加载预训练模型
model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')

# 编码句子为向量
sentences = ["帮我订机票", "我要买飞机票", "今天天气怎么样"]
embeddings = model.encode(sentences)

# 计算相似度
from sentence_transformers import util
similarities = util.cos_sim(embeddings[0], embeddings[1:])
# "帮我订机票" vs "我要买飞机票" → ~0.92
# "帮我订机票" vs "今天天气怎么样" → ~0.15
```

3.4.2 常用预训练模型

模型名	语言	维度	特点
<code>all-MiniLM-L6-v2</code>	英文	384	轻量快速，英文基线首选
<code>all-mpnet-base-v2</code>	英文	768	英文效果最佳
<code>paraphrase-multilingual-MiniLM-L12-v2</code>	多语言	384	支持中文，轻量
<code>shibing624/text2vec-base-chinese</code>	中文	768	中文专用，效果较好

新手解释 - 向量维度：向量维度越高，能编码的语义信息越丰富，但存储和计算开销也越大。384维是轻量级选择，768维是标准选择。实际选择取决于精度和速度的权衡。

3.4.3 微调训练

```
from sentence_transformers import SentenceTransformer, InputExample, losses

model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')

# 准备训练数据
train_examples = [
    InputExample(texts=["帮我订机票", "我要买飞机票"], label=1.0),    # 正例
    InputExample(texts=["帮我订机票", "今天天气怎么样"], label=0.0), # 负例
]

# 选择损失函数
train_dataloader = ...
train_loss = losses.CosineSimilarityLoss(model)

# 训练
model.fit(
    train_objectives=[(train_dataloader, train_loss)],
    epochs=3,
    warmup_steps=100
)
```

新手建议：Sentence Transformers 大幅降低了文本匹配的入门门槛。建议新手先用预训练模型跑通流程，再根据业务数据微调。

Part 4 基于LLM的文本匹配

4.1 Prompt Engineering：零样本方式

无需训练，通过 Prompt 描述任务，让 LLM 直接输出相似度判断。

模板 A：输出相似度分数

请判断以下两段文本的语义相似度。
输出 0~1 的浮点数，1 表示完全相同。

文本A: {text_a}
文本B: {text_b}

相似度分数:

模板 B：输出标签

判断以下两句话是否表达相同意思。
只输出"相同"或"不同"，不解释。

句子1: {text_a}
句子2: {text_b}

答案:

Prompt 设计要点：

- 明确输出格式，避免自由发挥
- 添加约束词（"只输出""不解释"）

零样本方式的特点：

维度	内容
优势	零标注成本，冷启动首选；借助 LLM 强大的语言理解能力；灵活应对多样的匹配标准；快速 POC 验证业务可行性
局限	推理慢（秒级），成本高；不适合大规模在线检索；输出格式不稳定，需后处理；效果依赖 Prompt 质量和 LLM 能力
适用场景	标注数据为零的冷启动；低频/离线评测任务；对延迟不敏感的人工辅助决策；需求快速变化

新手解释 - POC (Proof of Concept)： 概念验证，用最小成本验证一个想法是否可行。在NLP 项目中，先用LLM零样本方式快速验证"文本匹配能否解决我的业务问题"，确认可行后再投入资源训练专用模型。

4.2 Few-shot：示例驱动的文本文本匹配

在 Prompt 中给出 2~4 个标注示例，帮助 LLM 理解具体业务匹配标准。

Few-shot Prompt 示例：

你是一个意图匹配专家。判断用户输入
是否与意图描述匹配（是/否）。

示例1：
输入: "帮我订明天去上海的机票"
意图: "航班预订"
答案: 是

示例2：
输入: "查一下明天天气"
意图: "航班预订"
答案: 否

现在请判断：
输入: "{query}"
意图: "{intent}"
答案:

关键：示例要覆盖正负例 + 边界情况，2~4 个最佳，太多反而降低精度。

为什么 Few-shot 有效？

原因	解释
校准匹配标准	业务对"相似"的定义往往和通用理解不同，示例帮助LLM对齐
减少歧义	边界样本（正例中的难例/负例中的易例）明确判断边界
格式稳定性	示例统一了输出格式，减少后处理的不确定性

Few-shot vs Zero-shot：精度通常提升 5~15%，代价是 token 消耗增加；示例占 Prompt 的 30~50% 时效果最佳。

新手解释 - Zero-shot vs Few-shot：

- **Zero-shot（零样本）：**不给任何示例，直接让模型判断。就像让一个新员工直接干活，不给参考。
- **Few-shot（少样本）：**给几个示例，让模型"照猫画虎"。就像给新员工看几个案例再让他做。

4.3 LLM方式的定位与蒸馏落地

LLM 核心价值：在特定场景提供补充，或作为小模型的教师。

LLM方式的定位：

场景	应用	是否推荐
冷启动	无标注数据，快速上线	推荐
长尾需求	小类目/罕见意图数据稀缺	推荐
评测基准	用LLM打分作为指标参考	推荐
实时检索	大规模在线召回	不推荐
高并发	QPS > 100 的生产接口	不推荐

新手解释 - QPS（Queries Per Second）：每秒查询数，衡量系统吞吐量。QPS > 100 意味着每秒要处理100+个请求，LLM推理通常需要几秒，无法满足。

LLM 数据生成 + 蒸馏小模型：

1. LLM 对业务文本对打分/生成三元组
 2. 以此为伪标签训练 BiEncoder
 3. 小模型具备 LLM 的业务理解，推理快 100x

新手解释 - 伪标签（Pseudo Label）：不是人工标注的真实标签，而是由模型（这里是LLM）自动生成的标签。虽然可能有噪声，但数量大、成本低，可以作为训练数据使用。

4.4 四类方法全景对比

方法	精度	速度	数据量	规模
编辑距离/词向量	低	极快	无	任意
BiEncoder	中高	快	中量	百万级
CrossEncoder	高	慢	中量	百~千
LLM Prompt	高	极慢	极少	小规模

4.5 选型建议

场景	推荐方案
无数据冷启动	LLM Zero/Few-shot
大规模召回	BiEncoder + FAISS
高精度小候选集	CrossEncoder 精排
数据少+精度要求高	LLM 生成数据 → 蒸馏
生产级大规模高精度	Bi + Cross 两阶段

总结：文本匹配技术选型全景

技术演进

传统方法 → 词向量时代 → 预训练时代 → LLM时代			
编辑距离	word2Vec	BiEncoder	Prompt/Few-shot
TF-IDF	Glove均值	CrossEncoder	蒸馏
BM25		SimCSE	Embedding模型

核心认知

- 1. 没有最好的方法，只有最合适的方法
- 2. 效率与精度是核心权衡，不可兼得
- 3. 工业实践：两阶段（召回+精排）是黄金组合
- 4. 数据是核心瓶颈：难负例挖掘 > 模型选型
- 5. LLM 是生产数据的好老师，而非直接部署

工业落地典型流程

LLM打标 → BiEncoder预训练 → 难负例挖掘 → Bi+Cross上线

文本匹配是 NLP 工程的基石。理解每类方法的适用场景与瓶颈，才能在实际业务中做出正确的技术选型。

附录：专业术语速查表

术语	英文	简要解释
文本匹配	Text Matching	判断两段文本语义是否相似/相关
编辑距离	Levenshtein Distance	将一个字符串变成另一个的最少操作次数
TF-IDF	Term Frequency-Inverse Document Frequency	词频×逆文档频率，衡量词的重要性
BM25	Best Matching 25	改进的TF-IDF，搜索引擎标准排序算法
Word2Vec	Word to Vector	将词映射为向量的方法，上下文相似的词向量接近
GloVe	Global Vectors	基于全局共现统计的词向量方法
BiEncoder	Bi-Encoder	双塔模型，两个编码器独立编码后比较
CrossEncoder	Cross-Encoder	交叉编码器，拼接后一起编码，交互更充分
对比学习	Contrastive Learning	拉近相似样本、推远不相似样本的学习方法
SimCSE	Simple Contrastive Learning of Sentence Embeddings	利用Dropout做数据增强的句子嵌入对比学习
Dropout	Dropout	训练时随机丢弃部分神经元，防止过拟合
余弦相似度	Cosine Similarity	两个向量夹角的余弦值，衡量方向相似性
NLI	Natural Language Inference	自然语言推理，判断句子间的蕴含/矛盾/中立关系
MLM	Masked Language Model	遮住部分词让模型预测，BERT的预训练方式
FAISS	Facebook AI Similarity Search	Facebook开发的向量相似度搜索库
HNSW	Hierarchical Navigable Small World	分层可导航小世界图，一种近似最近邻索引
ANN	Approximate Nearest Neighbor	近似最近邻搜索，牺牲少量精度换取速度
知识蒸馏	Knowledge Distillation	用大模型指导小模型训练，让小模型继承大模型能力
模型量化	Quantization	将模型参数从高精度压缩为低精度，减少存储和加速推理
Focal Loss	Focal Loss	对难分类样本加权的损失函数
TripletLoss	Triplet Loss	三元组损失，要求正例比负例更接近锚点

术语	英文	简要解释
InfoNCE	InfoNCE	对比学习常用损失，从干扰项中识别正例
NT-Xent	Normalized Temperature-scaled Cross Entropy	带温度参数的对比损失函数
难负例	Hard Negative	与锚点表面相似但语义不同的负例
In-batch negatives	In-batch Negatives	同一batch中其他样本自动作为负例
伪标签	Pseudo Label	由模型自动生成而非人工标注的标签
POC	Proof of Concept	概念验证，用最小成本验证想法可行性
QPS	Queries Per Second	每秒查询数，衡量系统吞吐量
Zero-shot	Zero-shot Learning	零样本学习，不给示例直接让模型判断
Few-shot	Few-shot Learning	少样本学习，给少量示例让模型学习
领域偏移	Domain Shift	训练数据与实际应用数据的分布差异
过采样/下采样	Over-sampling / Under-sampling	增加少数类/减少多数类样本以平衡数据
弱监督	Weak Supervision	使用不完美（有噪声）的标签进行训练
nprobe	nprobe	IVF索引的搜索参数，控制搜索的聚类桶数量
ef	ef	HNSW索引的搜索参数，控制搜索的邻居节点数量
温度参数	Temperature (τ)	控制模型输出分布的平滑程度， τ 越小越尖锐
margin	Margin	三元组损失中的安全间距，确保正负例之间有足够间隔
SBERT	Sentence-BERT	基于BERT微调的句子嵌入模型，生成高质量句子向量
ColBERT	Contextualized Late Interaction over BERT	晚期交互模型，保留token级向量用MaxSim计算相似度
MaxSim	Max-Similarity	ColBERT中的操作，对每个查询词找文档中最匹配词的相似度
Pooling	Pooling	将token向量合并为句子向量的方法（CLS/平均/最大池化）
向量归一化	L2 Normalization	向量除以其长度，使余弦相似度等价于点积
点积	Dot Product / Inner Product	两向量对应位相乘再求和，归一化后等价余弦相似度

术语	英文	简要解释
欧氏距离	Euclidean Distance	两向量间的直线距离，值越小越相似
MRR	Mean Reciprocal Rank	第一个正确结果排名倒数的均值
MAP	Mean Average Precision	各查询平均精度的均值，评估排序质量
NDCG	Normalized Discounted Cumulative Gain	考虑位置权重的排序质量指标
Hit@K	Hit at K	前K个结果中是否命中正确答案
Recall@K	Recall at K	前K个结果中正确答案被找到的比例
AUC	Area Under ROC Curve	ROC曲线下面积，衡量排序能力
Spearman	Spearman Correlation	排序相关性，衡量预测顺序与真实顺序的一致性
乘积量化	Product Quantization (PQ)	将高维向量分段量化压缩，大幅减少存储
IVF	Inverted File Index	倒排索引，先聚类再搜索相关桶
回译	Back Translation	翻译成另一语言再翻回来，用于数据增强
梯度累积	Gradient Accumulation	多个小batch梯度累加后更新，等效大batch训练
Sentence Transformers	Sentence Transformers	最流行的句子嵌入工具库，封装SBERT等模型训练推理
LCQMC	Large-scale Chinese Question Matching Corpus	哈工大中文问题匹配数据集
SNLI	Stanford Natural Language Inference	斯坦福自然语言推理数据集
STS-B	Semantic Textual Similarity Benchmark	语义文本相似度基准数据集
向量维度	Embedding Dimension	向量的维度数，常见384/768，越高信息越丰富但开销越大

Part 5 新手实战指南：从零搭建文本匹配系统

目标读者：新手 NLP 算法工程师，已阅读前面理论部分，准备动手实践。

学习目标：通过一个完整的中文语义匹配项目，掌握 BiEncoder、CrossEncoder 和 LLM 三种方法从数据到评估的全流程。

5.1 项目总览

本项目使用 **AFQMC (蚂蚁金融问句匹配)** 数据集，完成以下任务：

阶段	任务	对应文件
数据探索	统计分布、长度分析、标签比例	<code>src/explore_data.py</code>
表示型训练	BiEncoder + CosineEmbeddingLoss / TripletLoss	<code>src/train_biencoder.py</code>
交互型训练	CrossEncoder + CrossEntropyLoss	<code>src/train_crossencoder.py</code>
模型评估	阈值搜索、F1/AUC、相似度分布	<code>src/evaluate.py</code>
方法对比	BiEncoder vs CrossEncoder vs LLM	<code>src/compare_methods.py</code>
LLM零样本	DashScope API 直接判断	<code>src_llm/llm_compare.py</code>
LLM微调	LoRA 指令微调 (SFT)	<code>src_llm/train_sft.py</code>

5.2 环境准备

```
# 1. 安装依赖
pip install torch transformers scikit-learn tqdm matplotlib
pip install sentence-transformers rank-bm25

# 2. 下载中文BERT预训练模型
# 放到 pretrain_models/bert-base-chinese 目录
# 或使用 HuggingFace 路径: google-bert/bert-base-chinese

# 3. 下载数据集 (AFQMC)
python src/download_data.py
```

5.3 第一步：数据探索（理解你的数据）

这是新手最容易跳过的步骤，也是最重要的步骤。在动手训练模型之前，必须先看懂数据长什么样。

```
# 运行数据探索
python src/explore_data.py
```

输出关键信息：

- 训练集规模：~34,000 条句对
 - 标签分布：正例约31%，负例约69% (**不均衡!**)
 - 句子平均长度：~13 字 (中文短文本)
 - 长度差分布：正例对长度差异小，负例对差异大
- 新手要点：**看到标签不均衡 (31% vs 69%)，意味着模型可能倾向于预测"不相似"。后续评估时不能只看准确率，要关注 F1 和 AUC。

5.4 第二步：训练 BiEncoder（表示型模型）

BiEncoder 是工业界最常用的方案，适合大规模检索场景。

5.4.1 模型架构理解

```
# 核心代码（见 src/model.py）
class BiEncoder(nn.Module):
    def encode(self, input_ids, attention_mask, token_type_ids):
        out = self.bert(input_ids, attention_mask, token_type_ids)
        vec = self._pool(out.last_hidden_state, attention_mask) # 池化
        vec = self.dropout(vec)
        return F.normalize(vec, p=2, dim=-1) # L2归一化

    def forward(self, batch_a, batch_b):
        emb_a = self.encode(**batch_a) # 编码句子A
        emb_b = self.encode(**batch_b) # 编码句子B
        return emb_a, emb_b
```

关键设计决策：

- 池化策略：默认 mean pooling（句子相似度任务上优于 CLS）
- L2 归一化：encode() 输出归一化后，余弦相似度 = 点积，方便 FAISS 检索
- 层数限制：num_hidden_layers=4（快速验证用，上线建议用 12 层）

5.4.2 两种损失函数

方式一：CosineEmbeddingLoss（推荐入门）

```
python src/train_bienncoder.py --loss cosine
```

- 输入：（句子A，句子B，标签）三元组
- label=1（正例）→ 拉近到 $\cos \approx 1$
- label=0（负例）→ 推远到 $\cos < \text{margin}$ （默认0.3）

方式二：TripletLoss（检索场景推荐）

```
python src/train_bienncoder.py --loss triplet
```

- 输入：（锚点，正例，负例）三元组
- 要求： $\text{sim}(\text{锚点}, \text{正例}) > \text{sim}(\text{锚点}, \text{负例}) + \text{margin}$
- 更明确地告诉模型“相对远近”关系

5.4.3 训练结果解读

训练完成后，AFQMC 验证集上典型结果：

- F1 (CosineEmbeddingLoss)：约 0.68（3 epoch，4层BERT）
- F1 (TripletLoss)：约 0.65（同上）

新手要点：BiEncoder 训练时输出的是相似度分数（0~1），评估时需要阈值搜索来找最优分类阈值。这不同于普通分类任务直接取 argmax。

5.5 第三步：训练 CrossEncoder（交互型模型）

CrossEncoder 精度更高但更慢，适合精排场景。

```
python src/train_crossencoder.py
```

核心区别：

- BiEncoder：两句分别编码 → 向量相似度
- CrossEncoder：[CLS] 句子A [SEP] 句子B [SEP] → 全层交互 → 分类

```
# 核心代码（见 src/model.py）
class CrossEncoder(nn.Module):
    def forward(self, input_ids, attention_mask, token_type_ids):
        out = self.bert(input_ids, attention_mask, token_type_ids)
        cls_vec = self.dropout(out.last_hidden_state[:, 0, :])
        logits = self.classifier(cls_vec) # [B, 2] → 相似/不相似
        return logits
```

训练结果：AFQMC 验证集 F1 约 0.68（4层BERT），与 BiEncoder 接近但使用场景不同。

新手要点： CrossEncoder 评估直接 `argmax` 即可，不需要阈值搜索。但它无法预计算向量，不适合大规模检索。

5.6 第四步：模型评估与对比

```
# 运行完整对比
python src/compare_methods.py
```

对比维度：

维度	BiEncoder	CrossEncoder	LLM（零样本）
训练数据	需要	需要	不需要
推理速度	毫秒级	数十毫秒	秒级
可预计算	是	否	否
适用规模	百万级	百~千级	小规模
F1（AFQMC）	~0.68	~0.68	~0.56

新手要点： 为什么 CrossEncoder 和 BiEncoder 的 F1 差不多？因为 AFQMC 是短文本简单匹配任务，4层BERT的交互优势不明显。在长文本或难区分场景下，CrossEncoder 的优势会更显著。

5.7 第五步：LLM 零样本尝试

适用场景：无标注数据，冷启动。

```
python src_llm/llm_compare.py
```

核心思路：通过 Prompt 让 LLM（如通义千问）直接判断语义相似度。

Prompt 模板：

判断以下两句话是否表达相同意思。
只输出"相似"或"不相似"，不解释。

句子1: {text_a}
句子2: {text_b}
答案:

LLM 方式的定位:

- 冷启动首选: 零标注成本
- 不推荐大规模在线使用: 推理慢 (秒级), 成本高
- 最佳用途: **用 LLM 生成伪标签数据, 训练小模型**

5.8 完整实战流程总结

1. 数据探索
 - ├─ 标签分布 (正负比例)
 - ├─ 文本长度分布
 - └─ 长度差与标签的关系
2. 基线实验
 - ├─ BiEncoder + CosineEmbeddingLoss
 - ├─ BiEncoder + TripletLoss
 - └─ CrossEncoder + CrossEntropyLoss
3. 对比分析
 - ├─ 精度对比 (F1、AUC)
 - ├─ 速度对比 (推理时间)
 - └─ 相似度分布图 (正负样本分离程度)
4. 进阶优化
 - ├─ 全量12层BERT (vs 4层)
 - ├─ 难负例挖掘
 - ├─ LLM 零样本对比
 - └─ LoRA 微调小模型
5. 上线部署
 - ├─ BiEncoder 粗召回 + CrossEncoder 精排 两阶段流水线

5.9 新手常见问题 FAQ

Q1: 为什么我的模型 F1 很低?

检查清单:

- ☐ 数据预览: 标签分布正常吗? 正负样本均衡吗?
- ☐ 最大长度设置: `max_length=64` 覆盖了 99.9% 的句子吗?
- ☐ 学习率: `2e-5` 对于 BERT 微调通常合适
- ☐ 训练轮数: 3~5 个 epoch 通常足够, 过拟合会掉点
- ☐ 阈值搜索: BiEncoder 的评估是否做了阈值搜索?

Q2: BiEncoder vs CrossEncoder 怎么选?

你的场景	选择
需要从百万文档中检索 Top-K	BiEncoder
已有 Top-100 候选，需要精排	CrossEncoder
两者都需要	两阶段流水线
没数据，要快速验证	LLM 零样本

Q3：4层BERT和12层BERT差距大吗？

4层用于快速实验，12层精度更高。建议先用4层跑通流程，再用12层做最终模型。

Q4：如何提升模型效果？

优先级从高到低：

- 1. **难负例挖掘**（效果提升最大）
- 2. 增加训练数据（LLM生成伪标签）
- 3. 全量12层BERT
- 4. 调参（学习率、batch size、margin）

Q5：如何判断模型是否上线可用？

- 业务指标：满足产品定义的精度/召回阈值得上线
- 对比基线：至少优于 BM25 等传统方法
- 边界测试：对难例、边界case做人工评估

给新手 NLP 算法工程师的建议：

文本匹配是 NLP 的"基本功"，几乎所有 NLP 产品都会用到。建议的学习路径是：

- 1. 先跑通一个完整项目（如本项目的 AFQMC）
- 2. 理解 BiEncoder 和 CrossEncoder 的本质区别
- 3. 掌握评估指标的含义和适用场景
- 4. 再逐步深入到生产环境的部署、优化、监控

不要一上来就追求最好的模型，先做出来，再做好。